

Dependency Injection

Carsten Gips (HSBI)

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Motivation aus Tests: Ich kann nicht mocken

```
public class Lsf {  
    public double getGrade(String matrikelnummer, String modulName) {  
        return 1.7; // Dummy-Wert (und Dummy-Typ!)  
    }  
}
```

```
public class Student {  
    private final String matrikelnummer;  
    private final String name;  
  
    public Student(String matrikelnummer, String name) {  
        this.matrikelnummer = matrikelnummer; this.name = name;  
    }  
  
    public String printGradeFor(String modulName) {  
        Lsf lsf = new Lsf(); // Harte Kopplung!  
        double grade = lsf.getGrade(matrikelnummer, modulName);  
        return String.format("%s hat im Modul %s die Note %s erreicht", name, modulName, grade);  
    }  
}
```

Was ist eine “Dependency”?

Eine **Dependency** (Abhängigkeit) ist ein Objekt, das eine Klasse **benötigt**, um ihre Aufgabe zu erfüllen.

- Beispiele:
 - `Student` braucht etwas, das Noten liefert (das `Lsf`)
 - Ein `Dozent` braucht vielleicht einen `GradeSubmissionService`
 - Ein `Controller` braucht ein `Repository`
- Typischerweise:
 - Eine Abhängigkeit ist ein Feld in der Klasse
 - häufig mit `private final` => “Ohne dieses Objekt kann ich meine Arbeit nicht machen.”

Idee von Dependency Injection

Grundprinzip:

- Statt: Klasse erstellt ihre Abhängigkeiten selbst mit `new`
- Besser: Klasse bekommt ihre Abhängigkeiten von außen “injiziert”

Typische Formen:

- Konstruktor-Injektion
- Setter-Injektion
- Interface-basierte Injektion (z.B. über Frameworks)

Tip

Don't call `new` all over the place - ask for what you need.

Schritt 1: Abstraktion der Abhängigkeit

Wir führen zunächst ein neues Interface für Notenabfragen ein:

```
public interface GradeService {  
    double getGrade(String matrikelnummer, String modulName);  
}
```

Das alte `Lsf` verpacken wir jetzt in einen `GradeService`:

```
public class LsfGradeService implements GradeService {  
    private final Lsf lsf;  
  
    public LsfGradeService(Lsf lsf) {  
        this.lsf = lsf;  
    }  
  
    @Override  
    public double getGrade(String matrikelnummer, String modulName) {  
        return lsf.getGrade(matrikelnummer, modulName);  
    }  
}
```

Schritt 2: Student mit Dependency Injection

Nun erweitern wir `Student` und eliminieren das feste `new Lsf()`. Stattdessen übergeben wir ein Objekt vom Typ `GradeService` im Konstruktor ("Konstruktor-Injektion"):

```
public class Student {  
    private final String matrikelnummer;  
    private final String name;  
    private final GradeService gradeService; // Dependency  
  
    // Dependency Injection  
    public Student(String matrikelnummer, String name, GradeService gradeService) {  
        this.matrikelnummer = matrikelnummer;  
        this.name = name;  
        this.gradeService = gradeService;  
    }  
  
    public String printGradeFor(String modulName) {  
        double grade = gradeService.getGrade(matrikelnummer, modulName);  
        return String.format("%s hat im Modul %s die Note %s erreicht", name, modulName, grade);  
    }  
}
```

Schritt 3: Verdrahtung im "Composition Root"

Wo kommen die Objekte her?

```
public class Demo {  
    public static void main(String[] args) throws InterruptedException {  
        // Echtes LSF  
        Lsf lsf = new Lsf();  
        GradeService gradeService = new LsfGradeService(lsf);  
  
        Student alex = new Student("123456", "Alex Muster", gradeService);  
        alex.printGradeFor("Programmieren 2");  
    }  
}
```

Alternative Implementierung (z.B. Demo/Test):

```
public class DummyGradeService implements GradeService {  
    @Override  
    public double getGrade(String matrikelnummer, String modulName) {  
        return 1.0; // immer sehr gut :)  
    }  
}
```

DI und Testbarkeit (JUnit)

JUnit-Test mit Fake-Implementierung (hier als Lambda-Ausdruck):

```
class StudentTest {  
    @Test  
    void createGradeMessage_usesInjectedGradeService() {  
        // given  
        GradeService fake = (_, modulName) -> modulName.equals("Programmieren 2") ? 1.3 : 4.0;  
        Student student = new Student("123456", "Alex Muster", fake);  
  
        // when  
        String message = student.printGradeFor("Programmieren 2");  
  
        // then  
        assertEquals("Alex Muster hat im Modul Programmieren 2 die Note 1.3 erreicht", message);  
    }  
}
```

- Eine Klasse soll **nicht selbst** ihre Abhängigkeiten mit `new` erzeugen
- Stattdessen soll sie **deklarieren**, was sie braucht (z.B. im Konstruktor)
- Abhängigkeiten werden über **Abstraktionen** (Interfaces) modelliert
- Konkrete Implementierungen werden von außen “injiziert”:
 - in kleinen Programmen durch Sie selbst (z.B. in `main`)
 - in größeren durch DI-Container/Frameworks



Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

